



Advanced MPI

I/O

2026

André Pereira

Conventional POSIX I/O



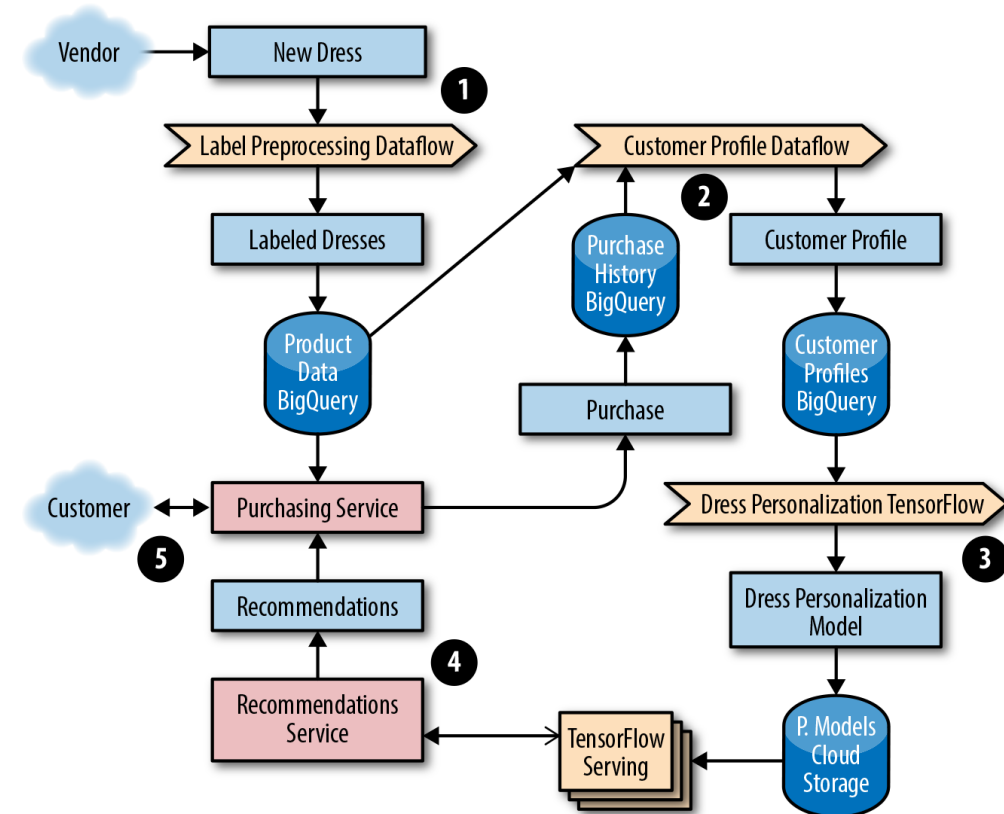
- POSIX (Portable Operating System Interface) defines the standard UNIX file access API
- In sequential programs, this works perfectly
 - The OS buffers data in RAM and flushes it to the local disk efficiently
- Not adequate for distributed-memory programs
 - When using these sequential APIs over a network to access a shared Parallel File System (PFS), severe scaling bottlenecks emerge

```
FILE *fp = fopen("data.bin", "w");  
  
// Seek to a specific byte offset  
fseek(fp, rank * SIZE, SEEK_SET);  
  
// Write the data sequentially  
fwrite(buffer, sizeof(double), SIZE, fp);  
  
fclose(fp);
```

Pattern 1: File-Per-Process (N-to-N)



- The most naive parallel approach
 - Each MPI rank creates its own unique file (e.g., data_0.bin, data_1.bin)
- PFSs (like Lustre or GPFS) use dedicated Metadata Servers (MDS)
 - Track file names, permissions, and exact locations across the cluster
- At scale, 10,000 ranks simultaneously calling `fopen` creates a massive DDOS-like "Metadata Storm"
 - Crashing the MDS and bringing the supercomputer's storage to a halt
 - Happened in Deucalion (guess what workload)



Pattern 2: Single Shared File (N-to-1)



POSIX Strict Consistency

- Avoid MDS storm
 - Often, all ranks open a single shared file, using `fseek` to find their offset
 - However, POSIX guarantees strict read/write consistency
 - To prevent data corruption, the OS strictly enforces distributed locking across the network

Serialization & Thrashing

- Locks force parallel writes to occur one at a time (serialisation).
 - Disk hardware thrashes wildly, trying to physically seek thousands of different fragmented offsets
 - High impact on total I/O bandwidth



- POSIX was designed for a single-node
 - Local multitasking, not exascale distributed computing
- Enforcing byte-level consistency across a vast network destroys parallel bandwidth
 - High-performance storage hardware requires coordinated, large, contiguous block writes
 - POSIX cannot express this natively



- MPI I/O solves the exascale storage bottleneck
 - Allows the MPI library to coordinate disk access over the high-speed network
 - Coordination **before** hitting the physical disks
- It acts as a powerful abstraction layer
 - Maps software communication patterns directly to the architecture of the PFS



10k

**Independent Disk
Writes**

- Assume 10,000 processes
 - Each call is a 1MB write of a fragmented matrix
 - PFS will experience extreme lock contention
- Disks will thrash
 - Seek to 10,000 different offsets simultaneously
 - Results in abysmal bandwidth and potential system timeouts



Phase 1: Network Aggregation

- MPI designates a few ranks as *I/O Aggregators*
- All ranks send their small, fragmented data pieces to these *Aggregators* first

Phase 2: Disk Write

- The *Aggregators* assemble these pieces into contiguous blocks in memory
- Perform a few large, sequential writes to the file system
- Disk seeks are minimised



- The file handle
 - Similar to `FILE*` in standard C
 - MPI I/O introduces the `MPI_File` opaque object
 - A file opened collectively by a communicator, retaining global state for the group
- Collective initialisation
 - `MPI_File_open`
 - Must be called by **all** processes in the specified communicator simultaneously
 - This establishes the coordinated access required for parallel operations

File Open and Close



- Opening a file requires
 - a communicator
 - a filename
 - access modes
 - `MPI_Info` object
- The `MPI_Info` object is critical for performance
 - Allows passing hardware-specific hints (such as striping size) directly to the file system

```
MPI_File fh;
int mode = MPI_MODE_CREATE |
MPI_MODE_WRONLY;

// Collective call over MPI_COMM_WORLD
MPI_File_open(MPI_COMM_WORLD, "data.bin",
              mode, MPI_INFO_NULL, &fh);

// ... perform parallel I/O ...

// Collective close
MPI_File_close(&fh);
```

MPI_File Access Modes



- **MPI_MODE_RDONLY**: read-only access.
- **MPI_MODE_WRONLY**: write-only access.
- **MPI_MODE_RDWR**: read and write access.
- **MPI_MODE_CREATE**: create the file if it does not exist.
- **MPI_MODE_APPEND**: set initial position of all file pointers to the end of the file.
- **MPI_MODE_EXCL**: throw an error if creating a file that already exists.

```
MPI_File fh;
int mode = MPI_MODE_CREATE |
MPI_MODE_WRONLY;

// Collective call over MPI_COMM_WORLD
MPI_File_open(MPI_COMM_WORLD, "data.bin",
              mode, MPI_INFO_NULL, &fh);

// ... perform parallel I/O ...

// Collective close
MPI_File_close(&fh);
```

The Concept of File Views



- A *View* acts as a template or a window into a file
 - No calculation of byte offsets manually
 - Each rank tells MPI exactly which parts of the file it owns
- The process only sees data that matches its view
 - A linear array written automatically maps to a strided write operation on the disk



- `MPI_Offset disp`
 - The number of bytes to skip from the beginning of the file
 - Often used to skip a global file header
- `MPI_Datatype etype`
 - The basic unit of data access (e.g., `MPI_DOUBLE`)
 - All file offsets are internally calculated in multiples of this type
- `MPI_Datatype filetype`
 - A derived datatype specifying the exact pattern of data layout in the file (e.g., a strided subarray)
 - This filters out the bytes belonging to other ranks

Setting a File View



- Using `MPI_Type_create_subarray`
 - A process can define its local chunk of a global grid
- The filetype describes how this chunk fits into the global file
- Writing to offset 0 writes to the correct location in the file

```
MPI_Offset disp = 0; // No header to skip

// Tell MPI this rank only sees the
elements
// defined by 'subarray_type'
MPI_File_set_view(fh, disp, MPI_DOUBLE,
                  subarray_type, "native",
                  MPI_INFO_NULL);
```

Data Access - Three Positioning Methods



- **Explicit offsets**
 - The most robust method.
 - Each rank explicitly states exactly where in the file it is writing or reading.
 - Highly favoured for reproducible science.
- **Individual pointers**
 - Each process maintains its own file pointer.
 - A write operation advances only the local pointer.
 - Good for independent contiguous writes.

Data Access - Three Positioning Methods



- **Shared pointers**
 - All ranks share a single global pointer.
 - Writing advances the pointer for everyone.
 - Useful for asynchronous parallel logging.

Explicit Offsets – MPI_File_write_at



- Explicit offsets are heavily preferred
 - Stateless
 - Order-independent
 - Reproducible across runs
- Calculate the exact offset
 - Based on rank and local array size
 - Use functions suffixed with `_at`

```
// Calculate exact offset based on rank  
MPI_Offset my_offset = rank *  
local_array_size;  
  
// Write 'count' elements starting at  
my_offset  
// (Offsets are in units of 'etype')  
MPI_File_write_at(fh, my_offset,  
local_buffer,  
count, MPI_DOUBLE,  
&status);
```



- If not using explicit offsets, rely on MPI managing the pointers.
- Shared pointers require atomic operations
 - To ensure consistency across the network
 - It can limit scalability
 - But it is necessary for unstructured data

```
// Individual Pointer Write
// Pointer advances by 'count' for THIS
rank only
MPI_File_write(fh, buf, count,
               MPI_DOUBLE, &status);

// Shared Pointer Write
// Global pointer advances by 'count' for
ALL
MPI_File_write_shared(fh, buf, count,
                     MPI_DOUBLE, &status);
```

Collective I/O API (*_all)



- Trigger the two-phase optimisation (slide 8)
 - Use functions with the `_all` suffix
- **Crucial**
 - All processes in the communicator
 - Open the file
 - Call the collective functions together
 - Even if they write 0 bytes

```
// INDEPENDENT (Slow for fragmented data)
MPI_File_write(fh, buf, count, type,
&status);

// COLLECTIVE (Fast, uses 2-phase
aggregation)
MPI_File_write_all(fh, buf, count, type,
&status);

// COLLECTIVE with Explicit Offset
MPI_File_write_at_all(fh, offset, buf,
count,
                                type, &status);
```



- Similar to `MPI_Isend`
- Non-Blocking I/O initiates a disk write and returns `MPI_Request` handle
 - Without stalling the CPU
- Perfect for **Asynchronous Checkpointing**
 - A simulation can snapshot its state to disk in the background
 - CPU continues to the next simulation time step
 - Hides the latency of the file system

Non-Blocking API (*_iwrite)



- Start the write using an `_iwrite` function
 - Meanwhile, perform an independent computation
 - Later, wait for the I/O to hit the disk
- Do not modify the memory buffer
 - Only after the non-blocking write is complete

```
MPI_Request req;

// Initiate write operation in the
// background
MPI_File_iwrite_at(fh, offset, buf, count,
                  MPI_DOUBLE, &req);

// ... compute next time step ...
compute_heavy_math();

// Wait for the I/O to safely hit the disk
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Performance Tuning Hints



Hint	Purpose	Value Example
<code>striping_factor</code>	Number of physical storage targets the file spans	16 or -1 (all)
<code>striping_unit</code>	The size of the chunks distributed across disks	1048576 (1 MiB)
<code>cb_nodes</code>	Number of <i>Aggregator</i> nodes to use during collective I/O	32
<code>cb_buffer_size</code>	Amount of RAM aggregators can use to stage data	16777216 (16 MiB)

Stencil I/O Case Study – Datatype Setup



- Avoid rank 0 running out of memory, gathering a massive grid
 - define a 2D Subarray datatype
- This tells MPI where the local $N \times N$ grid belongs inside the global $G \times G$ domain

```
int global_sizes[2] = {G, G};
int local_sizes[2]  = {N, N};
int starts[2]       = {my_row * N, my_col * N};

MPI_Datatype subarray;
MPI_Type_create_subarray(2, global_sizes,
                        local_sizes,
                        starts,
                        MPI_ORDER_C,
                        MPI_DOUBLE,
                        &subarray);
MPI_Type_commit(&subarray);
```

Stencil I/O Case Study – Writing



- Set the file view using the new subarray datatype
- Collective write automatically handles
 - The offset math
 - Placing data in the strided locations of the shared output file

```
// Set the view using the Subarray
MPI_File_set_view(fh, 0, MPI_DOUBLE,
subarray,
                    "native", MPI_INFO_NULL);

// Write collectively (bypasses Rank 0
gather!)
// 'grid' is just our local contiguous
array
MPI_File_write_all(fh, grid, N*N,
MPI_DOUBLE,
                    MPI_STATUS_IGNORE);

MPI_Type_free(&subarray);
```



- **Avoid File-Per-Process**
 - Never let thousands of ranks open thousands of files simultaneously.
- **Define Data Layouts**
 - Use `MPI_Datatype` to represent complex memory structures natively.
- **Use File Views**
 - Let MPI handle offset math.
 - It is safer and allows the library to optimise mapping.



- **Collective I/O Always**
 - Rely on `*_all` functions to utilise the two-phase optimisation for fragmented accesses.
- **Provide Info Hints**
 - Pass `MPI_Info` keys to tune your I/O for Lustre, GPFS, or specific network hardware.